

Practical Resources for Building Vision Language Models from Scratch in PyTorch

Based on your preference for hands-on learning and building models from scratch, I've found several excellent resources that will help you understand and implement VLMs in PyTorch. Here are the most practical and comprehensive options:

Video Tutorials (Recommended for Your Learning Style)

PaliGemma VLM Complete Implementation

The most comprehensive resource is a **5+ hour YouTube tutorial** that codes a complete Multimodal Vision Language Model from scratch using only Python and PyTorch^[1]. This tutorial covers:

- **Core Components:** Transformer model (Embeddings, Positional Encoding, Multi-Head Attention, Feed Forward Layer)
- **Vision Transformer:** Complete implementation with visual explanations
- **Advanced Concepts:** Contrastive learning (CLIP, SigLIP), Rotary Positional Embedding, Grouped Query Attention
- **Training Details:** KV-Cache, attention masks, weight tying, Top-P sampling
- **Numerical Stability:** Softmax and Cross Entropy Loss optimization

Repository: The tutorial includes a complete GitHub repository with code and detailed notes^[1].

Llama 4 Implementation with Vision Capabilities

Another practical video resource shows how to implement Llama 4 from scratch, including Vision Language Models and Mixture of Experts (MoE)^[2]. This covers modern VLM architectures and advanced techniques.

Hands-On Code Repositories

Seemore - Simplified VLM Implementation

The "**seemore**" project provides a hackable VLM implementation in pure PyTorch, consisting of^[3] ^[4]:

- **Image encoder** (vision component)
- **Multimodal projection module** (cross-modal alignment)
- **Decoder language model** (text generation)

This project is specifically designed as a simplified version of GPT-4 Vision capabilities, similar to open-source models like moondream 2 or LLaVA^[3]. The implementation is intentionally minimal to help understand the core concepts.

Key Features:

- Pure PyTorch implementation
- Character-level autoregressive language model (inspired by nanoGPT)
- Comprehensive blog post explaining the architecture^[5]

nanoVLM - Minimal Training Framework

nanoVLM is a lightweight toolkit specifically designed for training VLMs from scratch^[6] ^[7]. Key advantages:

- **Simplicity:** ~750 lines of human-readable code
- **Modularity:** Four decoupled components for easy customization
- **Performance:** Achieves 35.3% accuracy on MMStar benchmark with only 222M parameters
- **Hardware Efficiency:** Trains on a single GPU in 6 hours
- **Educational Focus:** Inspired by nanoGPT philosophy

Technical Architecture^[7]:

- Vision Backbone (~150 lines) - Feature extraction via SigLIP
- Language Decoder (~250 lines) - Text generation
- Modality Projection (~50 lines) - Cross-modal alignment
- VLM Integration (~100 lines) - Multimodal fusion logic

Educational Framework Resources

DeepSchool.ai VLM Tutorial

A structured tutorial focusing on vision-to-text conversion with practical PyTorch implementations^[8]. This resource covers the conversion of visual information into text and provides hands-on coding examples with proper attention mask handling and label processing.

PyTorch Official Resources

Several official PyTorch resources complement your VLM learning^[9] ^[10]:

- **Computer Vision fundamentals** with PyTorch
- **Transformer implementation guides**^[11] with complete building blocks
- **PyTorch Examples repository** with vision and language model implementations^[10]

Component-Level Understanding

To build VLMs from scratch effectively, you'll benefit from understanding each component:

Vision Transformers

The search results indicate that modern VLMs heavily use Vision Transformers for image processing^[1]. The PaliGemma tutorial specifically covers ViT implementation with visual tensor operation diagrams.

Cross-Modal Alignment

The modality projection module is crucial for aligning visual and textual representations^{[3] [6]}. Both seemore and nanoVLM provide clear implementations of this component.

Training Strategies

Key training considerations include^[6]:

- **Mixed precision training** with automatic FP16 optimization
- **Two learning rate approach**: Higher LR for projection modules, lower for pre-trained backbones
- **Dataset compatibility** with HuggingFace Datasets

Getting Started Recommendation

Given your PyTorch expertise and preference for building from scratch, I recommend this learning path:

1. **Start with the PaliGemma video tutorial**^[1] - comprehensive coverage with visual explanations
2. **Implement the seemore project**^{[3] [4]} - hands-on practice with a simplified architecture
3. **Experiment with nanoVLM**^[6] - understand modern training practices and evaluation
4. **Reference component tutorials**^{[8] [11]} for deeper understanding of specific modules

These resources collectively provide both theoretical understanding and practical implementation experience, perfectly aligned with your goals of understanding VLMs in depth through coding from scratch in PyTorch.



1. <https://www.youtube.com/watch?v=vAmKB7iPkWw>
2. <https://www.youtube.com/watch?v=yXbF-1n9wxs>
3. https://www.reddit.com/r/LocalLLaMA/comments/1cad54x/seemore_building_a_vision_language_model_from/
4. <https://github.com/AviSoori1x/seemore>
5. <https://huggingface.co/blog/AviSoori1x/seemore-vision-language-model>

6. <https://huggingface.co/blog/nanovlm>
7. <https://www.xugj520.cn/en/archives/lightweight-vision-language-models-pytorch-nanovlm.html>
8. <https://sachinruk.github.io/blog/2024-08-11-vision-language-models.html>
9. https://www.learnpytorch.io/03_pytorch_computer_vision/
10. <https://github.com/pytorch/examples>
11. <https://www.datacamp.com/tutorial/building-a-transformer-with-py-torch>